STRIPE PROJECT

2. OLAP Data System

Annexes

2.A. Kimball Analysis

> *Nicolas Pichon --- AIA RNCP 38777 / BC02 / D3 : OLAP Annexe A / v22 --- 2026/11/05.*

2.A.1. Méthode et périmètre

La méthode **Kimball** est une méthode de conception de système OLAP (cf. *The Data Warehouse Toolkit* par *Kimball*) : le point de départ n'est pas le schéma source, mais la problématique du métier. Le processus se déroule en quatre étapes, dans l'ordre suivant :

1. **Choisir le processus métier** : un événement qui se produit dans l'entreprise.
2. **Définir la granularité des faits** : ce que représente une ligne de la table de faits.
3. **Identifier les dimensions** : les axes d'analyse.
4. **Identifier les faits** : les mesures numériques agrégeables.

Le présent document couvre le processus complet pour l'ensemble des besoins analytiques exprimés dans le cahier des charges (BR2, DS2, D3, D8), en traçant chaque choix jusqu'à sa source dans le modèle OLTP (annexe 1.B). Les conventions de données (montants et devises, fuseaux horaires) et les règles de transformation (ETL) qui en découlent sont réunies dans l'annexe [D3-C].

2.A.2. Étape 1 : Les processus métiers

Six processus métiers sont identifiés, d'après les catégories de données analytiques du cahier des charges [D0] (DS2 : *Revenue Metrics*, *Customer Segmentation Data*, *Product Performance Metrics*, *Fraud Analysis Data*, *Compliance and Audit Logs*).

#	Processus métier	Événement déclencheur	Catégorie couverte
P1	Traitement d'un paiement	Une transaction est créée	Revenue Metrics, Product Performance, Customer Segmentation
P2	Évaluation du risque de fraude	Un score de fraude est calculé	Fraud Analysis Data
P3	Cycle de vie d'un abonnement	Un abonnement atteint une échéance	Revenue Metrics (récurrent)
P4	Accès et modification de données	Une consultation ou une modification est journalisée	Compliance and Audit Logs
P5	Traitement d'une demande de droit	Un client exerce un droit RGPD/CCPA	Compliance and Audit Logs
P6	Traitement d'un incident de sécurité	Un incident est détecté	Compliance and Audit Logs

Problématiques volontairement exclues : les remboursements (Refund) et rétro-facturations (ChargeBack) ne constituent pas des processus séparés. Ce sont des *attributs de l'issue* d'une transaction (P1), et non des événements analytiques autonomes (cf. BR2).

2.A.3. Étape 2 : Les grains

Le grain fixe ce qu'une ligne de fait représente, et ne doit jamais être mélangé avec un autre grain dans la même table (risque de double comptage lors des agrégations).

Fait	Grain	Type de table (Kimball)
`FactTransaction`	Une transaction individuelle	Fait transactionnel
`FactFraudEvent`	Une évaluation de risque de fraude	Fait transactionnel
`FactSubscriptionSnapshot`	Un abonnement, à la fin de chaque période de facturation	Instantané périodique
`FactAuditEvent`	Un événement d'audit (accès **ou** modification)	Fait transactionnel
`FactDataSubjectRequest`	Une demande de droit, mise à jour au fil du traitement	Instantané cumulatif
`FactSecurityIncident`	Un incident de sécurité, mis à jour au fil du traitement	Instantané cumulatif

Faits transactionnels et mises à jour. `FactTransaction` et `FactFraudEvent` sont insérés une fois, au grain ci-dessus. Seules quelques colonnes d'indicateurs sont mises à jour ensuite, parce que l'information arrive après l'événement (remboursement, litige, décision de fraude confirmée) : la liste exhaustive est donnée par la règle ETL-02 ([D3-C], 2.C.3.1). Les clés, `date_key` et les montants de la transaction ne changent pas.

Pourquoi `DataAccessLog` et `ChangeAuditLog` fusionnent-ils dans `FactAuditEvent` ?
Tables distinctes dans le système OLTP, elles correspondent au même type d'événement dans le système OLAP, car elles donnent la même réponse à la question : *"qui a fait quoi, quand, sur quoi"*. Un grain unique sert mieux le reporting consolidé de conformité, sans que cela contredise la séparation OLTP.

Pourquoi `FactDataSubjectRequest` et `FactSecurityIncident` sont-ils des instantanés cumulatifs et non des faits transactionnels classiques. Une demande de droit ou un incident traverse plusieurs jalons dans le temps (`received_at` --> `due_at` --> `fulfilled_at` ; `detected_at` --> `notified_at` --> `resolved_at`).
Le fait cumulatif Kimball est justement conçu pour ce cas : une ligne par demande/incident, mise à jour à chaque jalon franchi, avec une clé de date par jalon, ce qui permet de calculer directement des délais de traitement sans reconstruire l'historique à partir d'événements séparés.

Pourquoi `FactSubscriptionSnapshot` est-il un instantané périodique.
Un abonnement ne génère pas d'événement ponctuel quotidien. C'est son état (actif, montant, échéance) que l'on veut pouvoir suivre dans le temps. C'est l'usage typique de l'instantané périodique : une ligne par abonnement et par point de mesure (ici, la fin de chaque cycle de facturation, cohérent avec `Subscription.current_period_end`).

2.A.4. Étape 3 - Les dimensions

Les dimensions sont *conformées* : une même dimension (ex. `DimMerchant`) est partagée par plusieurs faits, avec les mêmes clés et attributs, pour permettre des analyses transverses (ex. comparer le revenu et les incidents de sécurité par marchand).
C'est la matrice de bus ci-dessous qui documente ce partage.

Dimension	Type SCD	Origine OLTP	Remarque
`DimDate`	Statique	Calendrier généré	Partagée par tous les faits
`DimMerchant`	SCD2	`Merchant` + `MerchantStatus` + `Country`	Historise les changements de statut/pays ; pays non nul, comme `Merchant.country_code`
`DimCustomer`	SCD2	`Customer` + `Country`	Historise le pays de résidence et le segment (ETL-10, [D3-C] 2.C.3.2) ; pays non nul depuis l'OLTP v36 (`Customer.country_code`)
`DimProduct`	SCD1	`Product`	Pas d'historisation : le prix courant suffit, le montant réel est déjà figé dans le fait
`DimPaymentMethod`	SCD1	`PaymentMethod` + `PaymentMethodType` + `CardNetwork`	Type et réseau de la carte (`network_code`), jamais le PAN
`DimPricingPlan`	SCD2	`PricingPlan`	Doit s'historiser : un plan corrigé rétroactivement ne doit pas réécrire le passé (cf. `Transaction.pricing_plan_id` et les règles MCD #22, #23, #24, #25 de l'OLTP) ; les frais fixes par devise (`PricingPlanFee`) ne sont pas repris
`DimCurrency`	Statique	`Currency`	
`DimGeography`	Statique	`Country` (référentiel) + `Transaction.ip_geolocation` (pays résolu à l'ETL) + `Transaction.device_type`	Pays de la **transaction**, pas celui du client ni du marchand. Dénormalisation assumée (deux notions combinées) ; membre « unknown » ; voir ci-dessous
`DimTransactionStatus`	Statique	`TransactionStatus`	Inclut `allows_refund` , `is_terminal`
`DimFraudRiskLevel`	Statique	`FraudRiskLevel`	Inclut `requires_manual_review` , `blocks_transaction`
`DimSubscriptionStatus`	Statique	`SubscriptionStatus`	Inclut `triggers_dunning`
`DimActor`	SCD2	`AuditActorType` (OLTP) + `actor_id` des journaux `DataAccessLog` / `ChangeAuditLog`	Historise le type d'acteur si réaffecté
`DimAuditAction`	Statique	`AuditAction`	Inclut `is_sensitive` , `requires_justification`
`DimRegulation`	Statique	`Regulation`	Inclut `response_deadline_days`
`DimRequestStatus`	Statique	`DataSubjectRequestStatus`	
`DimIncidentStatus`	Statique	`SecurityIncidentStatus`	
`DimReferenceExchangeRate`	Versionnée (annuelle)	Constante de référence, hors OLTP	Voir [D3-C] (2.C.1)

Dimensions indépendantes. Aucune dimension ne référence une autre dimension : seules les tables de faits portent des clés étrangères vers les dimensions (étoile pure, D3.2). `DimMerchant` étant en SCD2, une clé étrangère `DimCustomer` → `DimMerchant` désignerait une version du marchand et obligerait à versionner tous ses clients, produits et plans tarifaires à chaque changement du marchand. `DimCustomer` , `DimProduct` et `DimPricingPlan` portent donc `merchant_id` (clé métier de l'OLTP, sans lien) pour le filtrage RLS, et `DimReferenceExchangeRate` identifie la devise source par `from_currency_code` . Le marchand d'une ligne de fait reste `merchant_key` du fait ; l'ETL vérifie que le `merchant_id` du client, du produit ou du plan est celui de ce `merchant_key` .

Pourquoi `DimProduct` reste-t-elle en SCD1 alors que `DimMerchant` et `DimCustomer` sont en SCD2. Le montant réellement payé est déjà figé dans `FactTransaction.amount` au moment de la vente. Retrouver le prix catalogue *historique* d'un produit n'apporte rien que le fait ne porte déjà. `DimMerchant` / `DimCustomer` , en revanche, sont interrogées pour des attributs qui *évoluent indépendamment* du fait (statut, pays) et que l'on veut pouvoir dater précisément.

Lignée de `DimGeography`. La dimension décrit *où* et *avec quel appareil* la transaction a été initiée, et non *qui* est le client :

- **Pays** : `Transaction.ip_geolocation` contient l'adresse IP (IPv4/IPv6) de l'appelant. Le nom de la colonne est conservé dans l'OLTP, bien qu'il désigne une adresse IP et non un pays (la note de la colonne le précise). L'ETL la résout en pays avec une base GeoIP (version figée au chargement, pour qu'un rechargement donne le même résultat), puis rattache ce pays à `Country` pour obtenir le nom et la région.
- **Appareil** : `Transaction.device_type` (`mobile`, `desktop`, `tablet`).
- **Non résolu** : IP absente, privée, non résolue ou pays hors `Country` → membre par défaut « unknown » (`country_code = 'ZZ'`, `country_name = 'unknown'`, `region = 'unknown'`, `device_type = 'unknown'`). Les clés `geography_key` de `FactTransaction` et `FactFraudEvent` sont donc non nulles (une clé nulle ferait disparaître ces lignes des analyses par jointure interne).
- **RGPD** : l'adresse IP est une donnée personnelle ; elle n'est pas copiée dans l'OLAP, seul le pays l'est.
- **Pays du client et du marchand** : ils restent dans `DimCustomer.country_code` et `DimMerchant.country_code` (SCD2). Comparer ces pays au pays de la transaction permet d'identifier les transactions transfrontalières ou à pays discordant, utile en analyse de fraude.
- **Type SCD** : statique (SCD1). Une ligne par couple (pays, appareil) ; les changements de référentiel `Country` (nom, région) écrasent la valeur, sans historique.
- `FactAuditEvent` : `geography_key` est non nulle. Pour les accès (`DataAccessLog`), le pays est résolu depuis `source_ip`, l'adresse d'origine de la requête, comme pour `Transaction.ip_geolocation`. Pour les modifications (`ChangeAuditLog`, alimenté par CDC), l'OLTP v36 ne porte ni IP ni pays : ces lignes reçoivent le membre « unknown ». `device_type` vaut toujours `unknown` pour ce fait. Une analyse par pays ne couvre donc que les accès. Pour couvrir aussi les modifications, il faudrait porter l'adresse source dans `ChangeAuditLog` côté OLTP ; ce n'est pas retenu en l'absence de besoin.

2.A.5. Étape 4 : Les faits

Table de faits	Mesures	Nature
`FactTransaction`	Montants locaux en unités mineures : `amount_minor`, `fee_amount_minor`, `refund_amount_minor` ; montants en EUR : `amount_eur`, `fee_amount_eur`, `stripe_revenue_eur` (dérivée de `fee_amount_eur`), `refund_amount_eur`, `merchant_net_revenue_eur` (calculé) ; indicateurs : `is_refunded`, `is_chargeback`, `is_merchant_initiated` ; score de fraude aplati : `anomaly_score`	Additives sur toutes les dimensions sauf `is_*` (semi-additives) ; `anomaly_score` non additif (moyenne pertinente, somme non) ; les montants locaux ne s'additionnent qu'à devise constante
`FactFraudEvent`	`anomaly_score`, `is_confirmed_fraud`, `exposed_amount_eur` ; le niveau de risque, la revue manuelle et le blocage sont portés par `DimFraudRiskLevel` (`requires_manual_review`, `blocks_transaction`)	`anomaly_score` non additif (moyenne pertinente, somme non) ; `exposed_amount_eur` additif ; `is_confirmed_fraud` semi-additif
`FactSubscriptionSnapshot`	`mrr_eur` (revenu récurrent mensualisé, calculé, converti au taux de la devise de l'abonnement), `stripe_mrr_eur` (part de commission revenant à Stripe), `failed_attempt_count`, `is_active`, `days_until_renewal`	`mrr_eur` et `stripe_mrr_eur` semi-additifs (additif sur toutes les dimensions sauf le temps - sommer deux instantanés du même abonnement à deux dates n'a pas de sens)
`FactAuditEvent`	`was_authorized`, `had_justification` ; comptage par `COUNT(*)` (une ligne = un événement, pas de colonne `event_count`) ; la sensibilité et l'obligation de justification sont portées par `DimAuditAction` (`is_sensitive`, `requires_justification`)	Comptage entièrement additif
`FactDataSubjectRequest`	`processing_days` (calculé, `fulfilled_at − received_at`), `is_within_deadline` (le retard éventuel se déduit de `processing_days` et de `DimRegulation.response_deadline_days`, il n'est pas stocké)	Non additif (moyenne/max pertinents, jamais de somme)
`FactSecurityIncident`	`affected_record_count`, `detection_to_notification_hours`, `is_within_72h_deadline`	`affected_record_count` additif, les délais non additifs

Les règles de calcul de ces mesures et indicateurs sont réunies dans [D3-C] (2.C.3).

2.A.6. Matrice de bus (Kimball Bus Matrix)

Vue synthétique du partage des dimensions entre processus. Chaque case cochée est une dimension conformée, réutilisable telle quelle d'un fait à l'autre.

Dimension / Processus	P1 Transaction	P2 Fraude	P3 Abonnement	P4 Audit	P5 Droit RGPD	P6 Incident
DimDate	×	×	×	×	×	×
DimMerchant	×	×	×	×	×	×
DimCustomer	×	×	×	×	×	
DimProduct	×		×			
DimPaymentMethod	×					
DimPricingPlan	×					
DimCurrency	×					
DimReferenceExchangeRate	×		×			
DimGeography	×	×		×		
DimTransactionStatus	×					
DimFraudRiskLevel		×				
DimSubscriptionStatus			×			
DimActor				×		
DimAuditAction				×		
DimRegulation					×	
DimRequestStatus					×	
DimIncidentStatus						×

`DimDate` et `DimMerchant` traversent les six processus. C'est ce qui permettra, par exemple, de croiser revenu et incidents de sécurité par marchand et par mois sans jointure artificielle entre deux entrepôts séparés.

Références

Annexes

- [D3-C] OLAP Conventions and Transformation Rules

Spécifications

- [D0] Stripe Project Introduction